ARCHITECTURE DOCUMENT

PowerPulse

Adaptive Low-Power RISC-V SoC with Hardware Activity Profiling and Autonomous Peripheral Wake-Up

1. Block Diagram

The PowerPulse SoC is organised around the faculty-provided RISC-V processor core, which acts as the central processing unit and system master. The processor communicates with all memories and peripherals through a Bus Interconnect. The Hardware Activity Profiler (HAP) supplies idle indications to the Peripheral Power Management Controller (PPMC); the Autonomous Wake-Up Event Controller (AWEC) supplies wake-up requests to the PPMC; and the PPMC uses these inputs to manage the activity state of the selected peripherals. The RISC-V processor configures all policies and reads status registers through memory-mapped accesses.

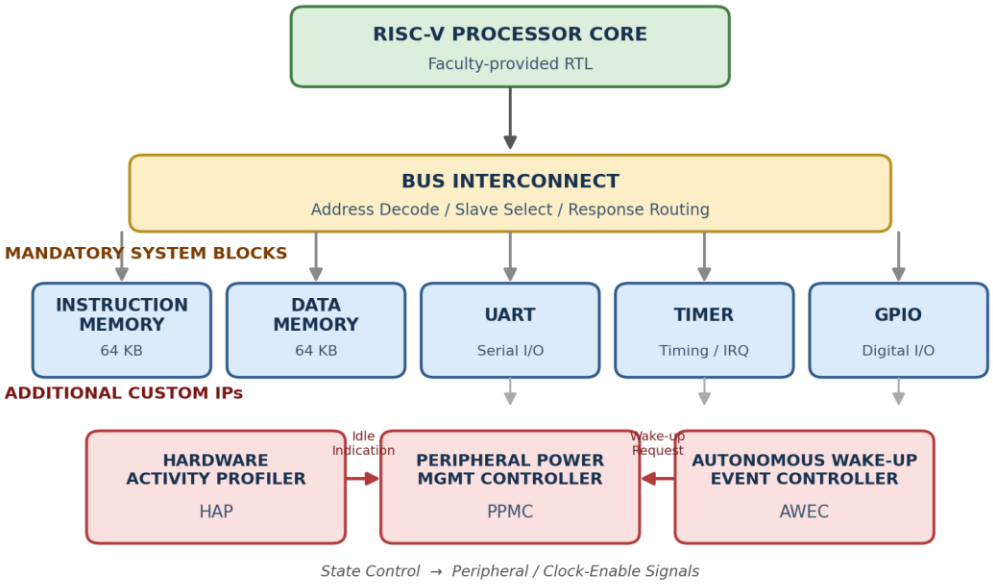


Figure 1. Top-level architecture — mandatory SoC infrastructure integrated with the HAP, PPMC and AWEC custom IP blocks.

1.1 Module Inventory

Module	Type	Role
RISC-V Processor Core	Mandatory / Provided	Executes the application and initiates memory-mapped transactions
Instruction Memory	Mandatory	Stores the program image executed by the processor
Data Memory	Mandatory	Stores runtime variables and application data
Bus Interconnect	Mandatory	Performs address decoding, slave selection and response routing
UART	Mandatory	Provides serial diagnostics and communication
Timer	Mandatory	Provides timing and programmable event generation
GPIO	Mandatory	Provides configurable digital I/O and external event detection

Module	Type	Role
Hardware Activity Profiler (HAP)	Additional Custom IP	Measures activity and generates programmable idle indications
Peripheral Power Mgmt. Controller (PPMC)	Additional Custom IP	Manages peripheral state and RTL-level enable control
Autonomous Wake-Up Event Ctrl. (AWEC)	Additional Custom IP	Detects enabled events and generates wake-up requests

2. Memory Map

The proposed SoC uses a flat 32-bit address space. Instruction and Data Memory occupy dedicated lower address regions, while the mandatory peripherals and additional IP blocks are located in a memory-mapped peripheral region. Each peripheral and additional IP is assigned an independent 256-byte register window, which simplifies address decoding and allows future register expansion.

Address Range	Block	Size	Access
0x0000_0000 – 0x0000_FFFF	Instruction Memory	64 KB	R/W*
0x0001_0000 – 0x0001_FFFF	Data Memory	64 KB	R/W
0x0002_0000 – 0x0002_00FF	UART	256 B	R/W
0x0002_0100 – 0x0002_01FF	Timer	256 B	R/W
0x0002_0200 – 0x0002_02FF	GPIO	256 B	R/W
0x0002_0300 – 0x0002_03FF	Hardware Activity Profiler	256 B	R/W
0x0002_0400 – 0x0002_04FF	Peripheral Power Mgmt. Controller	256 B	R/W
0x0002_0500 – 0x0002_05FF	Autonomous Wake-Up Event Controller	256 B	R/W
0x0002_0600 – 0xFFFF_FFFF	Reserved / Unmapped	—	—

** Instruction-memory write capability follows the final integration configuration of the faculty-provided RISC-V processor and memory wrapper.*

2.1 Additional IP Register Maps

Hardware Activity Profiler — Base Address: 0x0002_0300

Offset	Register	Description
0x00	HAP_CTRL	Enables or disables activity profiling
0x04	HAP_MANAGE_EN	Selects monitored peripheral activity sources
0x08	HAP_IDLE_TH	Programs the inactivity threshold
0x0C	HAP_IDLE_CNT	Reads the current inactivity count
0x10	HAP_STATUS	Reads activity and idle status

Peripheral Power Management Controller — Base Address: 0x0002_0400

Offset	Register	Description
0x00	PPMC_CTRL	Global enable for peripheral management
0x04	PPMC_POLICY	Selects automatic or software-directed policy
0x08	PPMC_PERIPH_EN	Controls peripheral-enable / clock-enable signals
0x0C	PPMC_STATE	Reads ACTIVE, IDLE or LOW-ACTIVITY state

Offset	Register	Description
0x10	PPMC_WAKE_MASK	Selects permitted wake-up sources

Autonomous Wake-Up Event Controller — Base Address: 0x0002_0500

Offset	Register	Description
0x00	AWEC_CTRL	Enables or disables wake-up monitoring
0x04	AWEC_ENABLE	Enables UART, GPIO and Timer wake sources
0x08	AWEC_STATUS	Reads pending wake-up event flags
0x0C	AWEC_EVENT_ID	Identifies the source of the captured event
0x10	AWEC_ACK	Acknowledges and clears a serviced event

3. Interface Specifications

The processor-facing bus wrapper follows the interface exposed by the faculty-provided RISC-V RTL configuration. The specification below defines the logical transaction roles required by the SoC and the three custom IP blocks; exact signal names will be adapted to the bus interface used in the faculty-provided processor RTL.

3.1 Common Memory-Mapped Slave Interface

Signal Function	Direction	Width	Description
System Clock	Input	1 bit	Common synchronous timing reference
Reset	Input	1 bit	Initialises internal and processor-visible state
Address	Master → Slave	32 bits	Byte address of the transaction
Select / Request	Master → Slave	1 bit	Indicates a valid request to the addressed target
Read / Write Control	Master → Slave	1 bit	Identifies transaction direction
Write Data	Master → Slave	32 bits	Carries data during write transactions
Read Data	Slave → Master	32 bits	Returns addressed register or memory data
Ready / Completion	Slave → Master	1 bit	Indicates transaction completion
Response / Error	Slave → Master	1+ bits	Reports transaction status where supported

3.2 Hardware Activity Profiler Interface

Inputs: system clock, reset, selected peripheral activity indications, and memory-mapped configuration/write/address signals. Outputs: peripheral activity status, inactivity count, programmable idle indication, memory-mapped read data and a completion/response signal.

Functional behaviour: whenever monitored peripheral activity occurs, the relevant inactivity count is reset. When no activity occurs for the configured threshold duration, the HAP asserts an idle indication, which the PPMC uses to transition the selected peripheral to LOW-ACTIVITY mode.

3.3 Peripheral Power Management Controller Interface

Inputs: system clock, reset, HAP idle indications, AWEC wake-up requests, and software configuration through memory-mapped registers. Outputs: peripheral-enable signals, clock-enable signals where applicable, current peripheral state and management status.

Functional behaviour: the PPMC implements the state transition ACTIVE → IDLE → LOW-ACTIVITY. A wake-up request causes the relevant peripheral to return via LOW-ACTIVITY → ACTIVE. The processor configures the management policy and reads the current state through memory-mapped registers.

3.4 Autonomous Wake-Up Event Controller Interface

Inputs: UART receive event, GPIO edge event, Timer compare event, wake-up source enable mask, clock and reset, and memory-mapped configuration. Outputs: wake-up request, pending event flags, wake-source identification and memory-mapped status information.

Functional behaviour — on a valid event: (1) the event is captured, (2) the source is identified, (3) a pending event status is stored, (4) a wake-up request is generated, (5) the PPMC restores the relevant peripheral to ACTIVE state, (6) software reads the event status, and (7) software acknowledges the event.

3.5 Inter-IP Control Relationship

Stage	Control Operation
Activity Monitoring	HAP observes selected peripheral activity indications
Idle Detection	HAP asserts an idle indication after the configured inactivity threshold
State Management	PPMC uses the idle indication to move the selected peripheral to LOW-ACTIVITY state
Event Detection	AWEC monitors enabled UART, GPIO and Timer events
Wake-Up Request	AWEC records the event and requests restoration of normal activity
State Restoration	PPMC returns the selected peripheral to ACTIVE state and software reads the status

4. Bus Protocol Details

The final bus protocol at the processor boundary will be the protocol implemented by the faculty-provided RISC-V integration wrapper. PowerPulse follows a single-master, memory-mapped architecture. For every valid processor transaction, the Bus Interconnect receives the address and control information, decodes the address, selects the corresponding memory or peripheral slave, routes write data to (or read data from) the selected slave, and returns completion and response information to the processor.

4.1 Transaction Phases

Phase	Processor / Master	Selected Slave
SETUP	Places Address, Write Control and Write Data; Select is asserted	Decodes the request and prepares the selected register access
ACCESS	Holds address / control information for the access phase	Performs the read/write operation and prepares response data
COMPLETE	Captures Read Data (for reads) and ends the transaction	Asserts Ready for completion, then returns to idle

4.2 Write Transaction Sequence

- The processor presents a valid address, write control and write data.
- The Bus Interconnect decodes the address and asserts the corresponding slave-select signal.
- The selected slave accepts the transaction and updates the addressed register or memory location.
- Completion and response information are returned through the Bus Interconnect to the processor.

4.3 Read Transaction Sequence

- The processor presents a valid address and read control.

- The Bus Interconnect decodes the address and selects the corresponding target slave.
- The selected slave returns the addressed register or memory value on its read-data path.
- The Bus Interconnect routes the selected read data and transaction response back to the processor.

4.4 Address Decoding and Response Selection

All implemented address regions are non-overlapping, so a valid mapped address activates one and only one slave-select path; during a read transaction, only the selected slave contributes data to the processor response. Addresses outside the implemented regions are treated as reserved or unmapped and return the default response supported by the final integration wrapper — an unmapped read returns zero data and completes the transaction, and an unmapped write modifies no slave state. This organisation simplifies verification and prevents ambiguity in peripheral selection.

4.5 Verification Plan

Verification is performed at IP level (inactivity counting, threshold detection, state transitions, register read/write behaviour, wake-up event detection, source masking, event capture and acknowledgement), at bus level (address decoding, slave selection, register access, response routing, unmapped-address handling), and at SoC level (processor application execution, memory-mapped transactions across all mandatory and additional IPs, idle-to-low-activity transitions, wake-up sequencing, software-visible status and UART diagnostic reporting).